

4339 Asset Pricing and Investments - Assignment 1

Tim Oberdammer (43356)

First Last (XXXXX)

First Last (XXXXX)

MSc in Finance, Stockholm School of Economics

September 14, 2026

1 Covariance:

The returns for two risky assets A and B are given for three equally likely states

Returns	State 1	State 2	State 3
Asset 1	-2%	-1%	3%
Asset 2	5%	3%	-5%

1.1 Expected Returns, standard deviation and correlation coefficient

The expected return $\mathbb{E}[r]$ is given by

$$\mathbb{E}[r] = \sum_{s=1}^S p_s r_s$$

where S is the total number of states and p_s is the probability of state s . Since all three states are equally likely, $p_s = \frac{1}{S}$ for all s , this simplifies to:

$$\mathbb{E}[r] = \frac{1}{S} \sum_{s=1}^S r_s$$

Substituting the values from the table yields

$$\mathbb{E}[R_A] = \frac{1}{3}(-2 - 1 + 3) = 0\%$$

$$\mathbb{E}[R_B] = \frac{1}{3}(5 + 3 - 5) = 1\%$$

The variance of returns is given by

$$\text{Var}(r) = \sigma^2 = \sum_{s=1}^S p_s (r_s - \mathbb{E}[r])^2$$

with the standard deviation being its square root

$$\sigma = \sqrt{\text{Var}(r)} = \sqrt{\sum_{s=1}^S p_s (r_s - \mathbb{E}[r])^2}$$

As above, p_s can be pulled out of the sum as the common factor.

Substituting the values from the table yields

$$\sigma_A = \sqrt{\frac{1}{3}[(-2-0)^2 + (-1-0)^2 + (3-0)^2]} = \sqrt{\frac{14}{3}} \approx 2.16\%,$$

$$\sigma_B = \sqrt{\frac{1}{3}[(5-1)^2 + (3-1)^2 + (-5-1)^2]} = \sqrt{\frac{56}{3}} \approx 4.32\%$$

The correlation coefficient is given by

$$\rho_{ij} = \text{Corr}[r_i, r_j] = \frac{\sigma_{ij}}{\sigma_i \sigma_j}$$

where the covariance $\text{Corr}[r_i, r_j]$ is defined as

$$\text{Cov}[r_i, r_j] = \mathbb{E}[(r_i - \mathbb{E}[r_i])(r_j - \mathbb{E}[r_j])]$$

Writing $\mu_i = \mathbb{E}[r_i]$ and $\mu_j = \mathbb{E}[r_j]$, which are constants, expanding the product and applying linearity of expectations gives

$$\begin{aligned} \sigma_{ij} &= \mathbb{E}[r_i r_j - \mu_j r_i - \mu_i r_j + \mu_i \mu_j] \\ &= \mathbb{E}[r_i r_j] - \mu_j \mathbb{E}[r_i] - \mu_i \mathbb{E}[r_j] + \mu_i \mu_j \\ &= \mathbb{E}[r_i r_j] - \mu_i \mu_j \\ &= \mathbb{E}[r_i r_j] - \mathbb{E}[r_i] \mathbb{E}[r_j] \end{aligned}$$

Substituting the values from the table yields

$$\sigma_{AB} = \frac{1}{3}[(-2)(5) + (-1)(3) + (3)(-5)] - (0)(1) = -\frac{28}{3} \approx -9.33 \text{ (\%}^2\text{)}.$$

$$\rho_{AB} = \frac{-\frac{28}{3}}{\sqrt{\frac{14}{3}} \cdot \sqrt{\frac{56}{3}}} = \frac{-\frac{28}{3}}{\sqrt{\frac{784}{9}}} = \frac{-\frac{28}{3}}{\frac{28}{3}} = -1$$

The assets are perfectly negatively correlated.

1.2 The fully invested Portfolio, made of A and B, with minimum variance

Since the portfolio is fully invested it follows that $w_B = 1 - w_A$ and the portfolio variance can be written as

$$\sigma_p^2 = w_A^2 \sigma_A^2 + (1 - w_A)^2 \sigma_B^2 + 2w_A(1 - w_A)\sigma_{AB}$$

To find the minimum Variance Portfolio we differentiate the variance function with respect to w_A ,

$$\frac{\partial \sigma_p^2}{\partial w_A} = 2w_A \sigma_A^2 - 2(1 - w_A) \sigma_B^2 + (2 - 4w_A) \sigma_{AB}$$

and set the derivative equal to zero

$$\begin{aligned} 0 &= 2w_A \sigma_A^2 - 2(1 - w_A) \sigma_B^2 + (2 - 4w_A) \sigma_{AB} \\ 0 &= w_A \sigma_A^2 - \sigma_B^2 + w_A \sigma_B^2 + \sigma_{AB} - 2w_A \sigma_{AB} \\ 0 &= w_A (\sigma_A^2 + \sigma_B^2 - 2\sigma_{AB}) - \sigma_B^2 + \sigma_{AB} \\ w_A^* &= \frac{\sigma_B^2 - \sigma_{AB}}{\sigma_A^2 + \sigma_B^2 - 2\sigma_{AB}} \end{aligned}$$

Substituting $\sigma_A^2 = \frac{14}{3}$, $\sigma_B^2 = \frac{56}{3}$ and $\sigma_{AB} = -\frac{28}{3}$

$$w_A^* = \frac{\frac{56}{3} + \frac{28}{3}}{\frac{14}{3} + \frac{56}{3} + \frac{56}{3}} = \frac{28}{42} = \frac{2}{3}, \quad w_B^* = \frac{1}{3}.$$

The resulting portfolio variance is

$$\sigma_p^2 = \frac{4}{9} \cdot \frac{14}{3} + \frac{1}{9} \cdot \frac{56}{3} + 2 \cdot \frac{2}{3} \cdot \frac{1}{3} \cdot \left(-\frac{28}{3}\right) = \frac{56}{27} + \frac{56}{27} - \frac{112}{27} = 0.$$

1.3 The set of all possible arbitrage portfolios when a risk-free bond with return 0% exists and a no-short sale constraint is imposed on A and B

The zero-cost portfolio therefore borrows at $r_f = 0\%$ and invests the money in the two risky assets. To begin with, we normalise the borrowed amount to one unit and

$$w_A + w_B = 1, \quad w_A, w_B \geq 0,$$

so that $w_B = 1 - w_A$ with $w_A \in [0, 1]$.

As the bonds return is 0, the portfolio return in state s is $r_{P,s} = w_A r_{A,s} + (1 - w_A) r_{B,s}$. Following the no-arbitrage condition this must yield a non-negative amount in every state, which allows us to set up three inequalities

$$\text{State 1: } 0 \leq -2w_A + 5(1 - w_A) = 5 - 7w_A \implies w_A \leq \frac{5}{7} \approx 0.7143,$$

$$\text{State 2: } 0 \leq -1w_A + 3(1 - w_A) = 3 - 4w_A \implies w_A \leq \frac{3}{4} = 0.75,$$

$$\text{State 3: } 0 \leq 3w_A - 5(1 - w_A) = 8w_A - 5 \implies w_A \geq \frac{5}{8} = 0.625$$

Since the first condition is stricter than the second one, only conditions (1) and (3) are binding. Combining the remaining two yields

$$\frac{5}{8} \leq w_A \leq \frac{5}{7} \quad \text{i.e.} \quad 0.625 \leq w_A \leq 0.7143$$

As there exists no w_A which satisfies all three conditions with equality simultaneously, i.e. $w_A = \frac{5}{7}$, $w_A = \frac{3}{4}$ and $w_A = \frac{5}{8}$, each portfolio earns a strictly positive return in at least one state.

By normalising to a single unit of borrowed money, we obtained the proportions of w_A and w_B . Scaling the position up or down by shorting more or less of the risk-free asset gives all possible arbitrage portfolios (w_A, w_B, w_f) as $x \cdot (w_A, 1 - w_A, -1)$ for any $x > 0$ and any $w_A \in [\frac{5}{8}, \frac{5}{7}]$.

2 Optimal Portfolio Allocation with two risky assets:

The expected return, standard deviation and the covariance of a corporate bond fund and an equity fund are given

Security	Expected Return	Standard Deviation	Covariance
Stock Fund	10%	20%	0.0002
Bond Fund	5%	8%	

2.1 Compute and plot the efficient frontier

To solve this we use `opt_portfolio_2asset.ipynb` with the values referring to the table above.

```
[156]: import numpy as np
import matplotlib.pyplot as plt

# Setup
# Basic inputs for bonds and stocks

# Mean
SB_mean = np.array([[0.1], [0.05]])

# Variance-Covariance Matrix
SB_Sigma = np.array([[0.2**2, 0.0002], [0.0002, 0.08**2]])

# Vector of ones
ones2 = np.ones((2, 1))
```

```
# Inverse of the variance-covariance matrix
SB_invSigma = np.linalg.inv(SB_Sigma)
```

```
[157]: # Define vectors for weights: Stock = (0,1)^t and Bond = (1,0)^t
E_w = np.array([[1], [0]]) # Bond
B_w = np.array([[0], [1]]) # Stock

# Create many portfolios as a combination of stock and bond

# Possible combinations of weights
y = np.arange(-1, 2.01, 0.01)

# Convert y to a 2D array where each row is a weight for a portfolio
y = y.reshape(1, -1)
port = np.dot(B_w, y) + np.dot(E_w, (1 - y))

# Compute expected returns
port_mean = np.dot(port.T, SB_mean)

# Compute var-covar matrix of portfolio returns (we will only use the variances
↳ on the main diagonal)
port_var = np.diag(np.dot(np.dot(port.T, SB_Sigma), port))
port_var = np.clip(port_var, 0, None) # guard against tiny negative values
↳ from float rounding
port_vol = np.sqrt(port_var).reshape(len(port_var), 1)
```

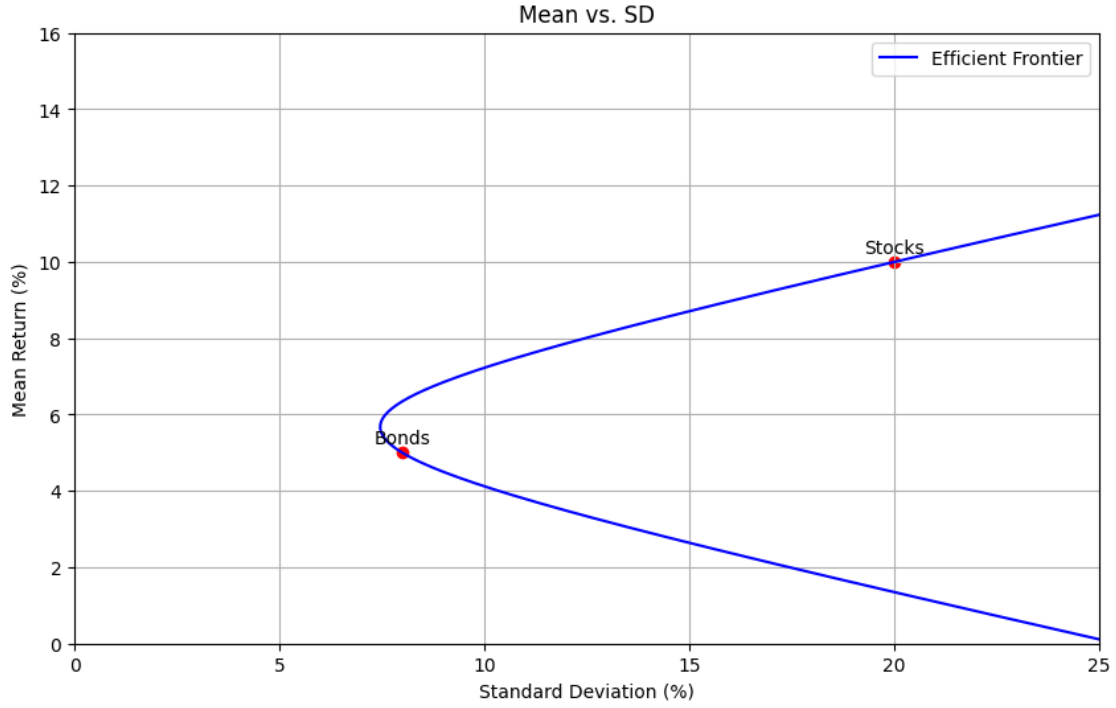
```
[158]: # Plot MV
plt.figure(figsize=(10, 6))
plt.plot(port_vol * 100, port_mean * 100, 'b-', label='Efficient Frontier')
plt.title("Mean vs. SD")
plt.xlabel("Standard Deviation (%)")
plt.ylabel("Mean Return (%)")
plt.xlim(0, 25)
plt.ylim(0, 16)

# Add points for stock and bond funds
plt.scatter(np.sqrt(np.diag(SB_Sigma)) * 100, SB_mean * 100, color='red')
SB_labels = ["Stocks", "Bonds"]

# Annotate points
for i, txt in enumerate(SB_labels):
    plt.annotate(txt, (np.sqrt(np.diag(SB_Sigma))[i] * 100, SB_mean[i, 0] *
↳ 100), textcoords="offset points", xytext=(0, 5), ha='center')

plt.legend()
plt.grid(True)
```

```
plt.show()
```



2.2 Compute the asset weights that make up the minimum variance portfolio (GMV)

Following *Section 1.3* the weights of the fully invested global minimum variance portfolio, with 2 risky assets, are

$$w_A^* = \frac{\sigma_B^2 - \sigma_{AB}}{\sigma_A^2 + \sigma_B^2 - 2\sigma_{AB}} \quad \text{and} \quad w_B^* = 1 - w_A^*$$

Substituting $\sigma_A^2 = 0.2^2$, $\sigma_B^2 = 0.08^2$ and $\sigma_{AB} = 0.0002$

$$w_A^* = \frac{0.08^2 - 0.0002}{0.2^2 + 0.08^2 - 0.0004} = \frac{31}{230} \approx 0.1349, \quad w_B^* = \frac{199}{230} \approx 0.8652.$$

The resulting portfolio variance is

$$\sigma_P^2 = \left(\frac{31}{230}\right)^2 0.04 + \left(\frac{199}{230}\right)^2 0.0064 + 2 \cdot \frac{31}{230} \cdot \frac{199}{230} \cdot 0.0002 \approx 0.005564$$

and the standard deviation is

$$\sigma_P \approx 0.074595 = 7.46\%$$

The expected return of the GMV portfolio is

$$\mu_P = \left(\frac{31}{230}\right) 0.10 + \left(\frac{199}{230}\right) 0.05 \approx 0.0567 = 5.67\%.$$

It can be seen that holding a certain amount of the riskier stock fund reduces the overall portfolio risk compared to the safer bond fund, $\sigma_P = 7.46\% < \sigma_B = 8\%$.

2.3 Assuming a risk-free asset, with $r_f = 0.03$, derive the tangent portfolio and the optimal CAL. Plot them on a graph with the efficient frontier.

The tangent portfolio maximizes the Sharpe Ratio S , which is defined as

$$S = \frac{\mathbb{E}[r_T] - r_f}{\sigma_T}$$

For the case with two risky assets let $g(w_A)$ be the excess return function and $h(w_A)$ be the variance function

$$\begin{aligned} g &= w_A r_A + (1 - w_A) r_B - r_f \\ h &= w_A^2 \sigma_A^2 + (1 - w_A)^2 \sigma_B^2 + 2w_A(1 - w_A) \sigma_{AB} \end{aligned}$$

As the tangency portfolio lies on the efficient frontier and is as such made up of only the two risky assets, we can use $w_B = 1 - w_A$.

To maximize the Sharpe Ratio we differentiate S with respect to w_A and set this FOC equal to 0

$$\begin{aligned} S(w_A) &= \frac{g}{\sqrt{h}} \\ S(w_A) &= g h^{-1/2} \end{aligned}$$

By applying the product rule we find the FOC

$$\begin{aligned} \frac{\partial S}{\partial w_A} &= g' h^{-1/2} + g \left(-\frac{1}{2}\right) h^{-3/2} h' \\ &= h^{-3/2} [g' h - \frac{1}{2} g h'] \end{aligned}$$

Now we can set the derivative equal to 0 and multiply by $h^{3/2}$, which gives

$$\begin{aligned} g' h - \frac{1}{2} g h' &= 0 \\ 2g' h &= g h' \end{aligned}$$

Substituting into $2g' h = g h'$, differentiating the remaining parts and simplifying, results in

$$w_A^T = \frac{(r_A - r_f) \sigma_B^2 - (r_B - r_f) \sigma_{AB}}{(r_A - r_f) \sigma_B^2 + (r_B - r_f) \sigma_A^2 - [(r_A - r_f) + (r_B - r_f)] \sigma_{AB}}$$

Substituting $r_A = 0.10$, $r_B = 0.05$, $r_f = 0.03$, $\sigma_A^2 = 0.2^2$, $\sigma_B^2 = 0.08^2$ and $\sigma_{AB} = 0.0002$

$$w_A^T = \frac{(0.07)0.0064 - (0.02)0.0002}{(0.07)0.0064 + (0.02)0.04 - (0.09)0.0002} = \frac{74}{205} \approx 0.3610, \quad w_B^T = \frac{131}{205} \approx 0.6390.$$

The optimal CAL is the linear function through the risk-free asset and the tangency portfolio T with r_f as the y-intercept and the Sharpe Ratio as the slope

$$CAL = r_f + \frac{E[r_T] - r_f}{\sigma_T} \sigma_{CAL} = 0.03 + 0.4276 \sigma_C,$$

The following code was adapted from 'opt_portfolio_2assets.ipynb' with the values referring to the results from above.

```
[159]: # Risk-free rate
rf = 0.03

# Excess return
SB_excess = SB_mean - rf

# Tangent portfolio weights
wT1 = (SB_Sigma[1, 1] * SB_excess[0] - SB_Sigma[1, 0] * SB_excess[1]) / \
      (SB_Sigma[0, 0] * SB_excess[1] + SB_Sigma[1, 1] * SB_excess[0] -
      ↪ SB_Sigma[0, 1] * np.sum(SB_excess))

# Ensure wT1 is a scalar
wT1 = wT1[0]

T_w = np.array([[wT1], [1 - wT1]])

# Mean and volatility of tangent portfolio (as scalars)
T_mean = np.dot(T_w.T, SB_mean).item()
T_vol = np.sqrt(np.dot(np.dot(T_w.T, SB_Sigma), T_w).item())

# Sharpe ratio
T_SR = (T_mean - rf) / T_vol

print(f"w_A = {wT1:.4f}, w_B = {1 - wT1:.4f}")
print(f"E[r] = {T_mean*100:.3f}%, sd = {T_vol*100:.3f}%, Sharpe = {T_SR:.4f}")

# Possible combinations of weights
yT = np.arange(-10, 10.01, 0.01)
yT = yT.reshape(1, -1)

# Create many portfolios using E and B
portT = np.dot(E_w, yT) + np.dot(B_w, (1 - yT))

# Compute expected returns
portT_mean = np.dot(portT.T, SB_mean)
```

```

# Compute var-covar matrix of portfolio returns (we will only use the variances,
↳on the main diagonal)
portT_var = np.diag(np.dot(np.dot(portT.T, SB_Sigma), portT))
portT_vol = np.sqrt(portT_var).reshape(len(portT_var), 1)

# Compute var-covar matrix of portfolio returns (we will only use the variances,
↳on the main diagonal)
portT_var = np.diag(np.dot(np.dot(portT.T, SB_Sigma), portT))
portT_vol = np.sqrt(portT_var).reshape(len(portT_var), 1)

```

```

w_A = 0.3610, w_B = 0.6390
E[r] = 6.805%, sd = 8.898%, Sharpe = 0.4276

```

```

[160]: plt.figure(figsize=(10, 6))
plt.plot(portT_vol * 100, portT_mean * 100, 'b-', label='Efficient Frontier')
plt.title("Mean vs. SD")
plt.xlabel("Standard Deviation (%)")
plt.ylabel("Mean Return (%)")
plt.xlim(0, 25)
plt.ylim(0, 16)

# Add points for Tangent portfolio
plt.scatter(T_vol * 100, T_mean * 100, color='green', marker='o')
plt.text(T_vol * 100, T_mean * 100, 'Tangent Portfolio', fontsize=9,
↳ha='center', va='bottom')

# Add points for stock and bond funds
plt.scatter(np.sqrt(np.diag(SB_Sigma)) * 100, SB_mean * 100, color='red',
↳marker='o')
SB_labels = ["Stocks", "Bonds"]
for i, txt in enumerate(SB_labels):
    plt.text(np.sqrt(np.diag(SB_Sigma))[i] * 100, SB_mean[i, 0] * 100, txt,
↳        fontsize=9, ha='center', va='bottom')

# Add the CAL to the picture

# Possible combinations of weights
yf = np.arange(0, 10.01, 0.01)

# Compute expected returns
CAL_mean = rf + yf * (T_mean - rf)

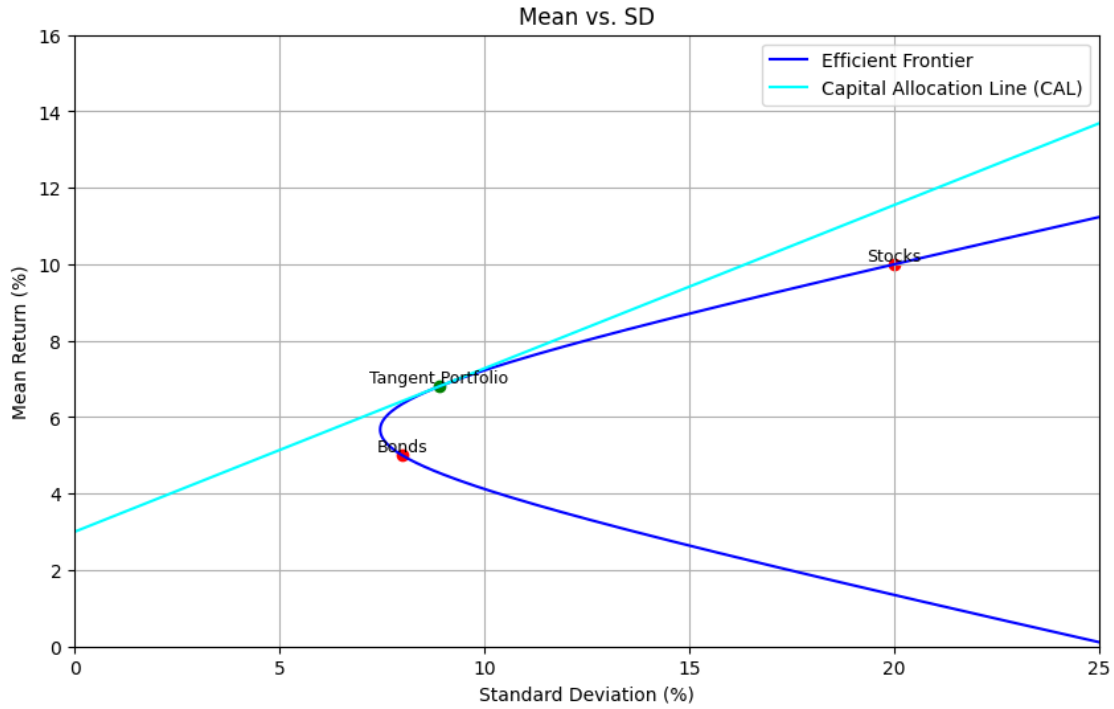
# Compute volatilities
CAL_vol = yf * T_vol

# Add CAL to the graph

```

```
plt.plot(CAL_vol * 100, CAL_mean * 100, color='cyan', label='Capital Allocation Line (CAL)')

plt.legend()
plt.grid(True)
plt.show()
```



2.4 The optimal portfolio for an investor with risk aversion parameter $a = 4$ and its certainty equivalent

The investor can now move along the CAL by splitting his investment between the risky tangency portfolio and the risk-free asset. Therefore, a risk averse investor maximizes his utility according to the following utility function

$$U^* = r_f + w^*(\mathbb{E}[r_T] - r_f) - \frac{a}{2}(w^*)^2\sigma_T^2$$

To find the maximum we differentiate with respect to w and set it equal to 0

$$\frac{dU}{dw} = (\mathbb{E}[r_T] - r_f) - a w \sigma_T^2 = 0 \quad \Rightarrow \quad w^* = \frac{(\mathbb{E}[r_T] - r_f)}{a \sigma_T^2}$$

Substituting the values yields

$$w^* \approx \frac{0.06805 - 0.03}{4(0.007918)} = 1.2014$$

To maximize his utility the investor takes on leverage by shortselling 20.14% of the risk-free asset and investing 120.14% into the tangency portfolio.

The certainty equivalent is the risk-free rate r_f^* that provides the investor with the same utility as the above portfolio. As a sole investment in a risk-free asset carries no risk (volatility), the utility function simplifies to

$$U^* = r_f^*$$

To find this rate we set the “risk-free utility function” equal to the general utility function from above

$$\begin{aligned} r_f^* &= r_f + w^*(\mathbb{E}[r_T] - r_f) - \frac{a}{2}(w^*)^2\sigma_T^2 \\ r_f^* &\approx 0.03 + 1.2014(0.06805 - 0.03) - \frac{4}{2}(1.2014)^2(0.007918) \\ r_f^* &\approx 0.05286 \end{aligned}$$

The investor is thus indifferent between his optimal risky portfolio and a safe return of 5.29%

2.5 Effects of changes to the risk aversion parameter a and to the risk-free rate r_f

As shown in Section 2.3, the tangency portfolio is not defined by the risk aversion parameter. Therefore increasing a to 8 has no effect on the stock-bond split that makes up the tangency portfolio.

However, it affects the position the investor takes along the CAL in order to maximise his utility. Following the optimal weight formula from Section 2.4, w^* is inversely proportional to a , and since neither $\mathbb{E}[r_T]$ nor σ_T depends on risk aversion, doubling a exactly halves the weight placed in the tangency portfolio

$$a = 2 \cdot 4 = 8 \quad \implies \quad w^* = \frac{1}{2}(1.2014) = 0.6007.$$

This is also known as the two fund separation theorem.

In contrast, changing the risk-free rate to $r_f = 0.04$ affects the Sharpe maximisation, which defines the tangency portfolio. The weight in stocks w_A rises from 36.1% to 49.6%, moving the tangency portfolio to nearly an even split between stock and bond.

This is because, while in absolute terms both assets lose an equal 1% of excess return, the bond loses more in relative terms. Its premium is halved from 2% to 1%, whereas the equity premium falls only from 7% to 6%. As the volatilities remain unchanged, the bond loses more of its return-to-risk ratio. Consequently the portfolio Sharpe maximisation favours the stock portion more.

3 Mean-Variance Frontier:

The following 6 risky assets are available to trade

	Asset 1	Asset 2	Asset 3	Asset 4	Asset 5	Asset 6
Expected Return (%)	8%	10%	9%	5%	12%	15%
Standard Deviation (%)	10%	12%	16%	7%	15%	20%

3.1 One asset that is dominated by another asset

An asset is dominated by another asset if it provides a lower expected return while having an equal or higher amount of risk or if it provides a lower or equal expected return while having an higher amount of risk.

This is the case for asset 3 and asset 2 where both inequalities hold strictly

$$\mathbb{E}[r_3] = 9\% < 10\% = \mathbb{E}[r_2] \quad \text{and} \quad \sigma_3 = 16\% > 12\% = \sigma_2$$

As such, asset 2 dominates asset 3 by offering a higher expected return while carrying a lower amount of risk. It should be noted, that in portfolio allocation this does not necessarily result in the exclusion of asset 3, as the covariance between the assets is also a critical measure.

3.2 Assets that lie on the efficient frontier given a no-short sale constraint

Following the no-short sale constraint, the expected portfolio return for N assets, which are indexed by i , is

$$\mathbb{E}[r_P] = \sum_{i=1}^N w_i r_i \quad \sum_{i=1}^N w_i = 1 \quad w_i \geq 0.$$

When a no-shortsale constraint is imposed, the highest achievable portfolio return is the portfolio that is fully invested in the highest returning asset.

$$\mathbb{E}[r_{P,max}] = 1 \cdot r_{i,max}$$

From the table above, the highest returning asset is asset 6.

The efficient frontier consists of all constructable portfolios that provide the highest level of return for a given level of risk. Since no other asset achieves a higher return and asset 6 is the only one achieving 15%, there exists no other portfolio that offers at least the same return for a lower or equal amount of risk. Asset 6 must therefore lie on the efficient frontier regardless of its volatility of 20%.

4 Retirement Portfolios:

Analysis of the long-term historical performance of the following retirement portfolios from the perspective of a US-investor:

- Equity portfolio: 100% S&P 500
- 80/20 portfolio: 80% S&P 500, 20% 10-year U.S. Treasury bonds
- 60/40 portfolio: 60% S&P 500, 40% 10-year U.S. Treasury bonds
- 40/60 portfolio: 40% S&P 500, 60% 10-year U.S. Treasury bonds
- Target date fund: a fund that starts with 90% S&P 500 and 10% 10-year U.S. Treasury bonds and gradually shifts to 40% S&P 500 and 60% 10-year U.S. Treasury bonds over 35 years.

The following analysis uses the dataset “Historical Returns on Stocks, Bonds and Bills: 1928-2024” curated by Aswath Damodaran. This dataset was selected as it gives both the total equity return (appreciation and dividends) and the total bond returns (coupons and price appreciation). The dataset was retrieved from https://pages.stern.nyu.edu/~adamodar/New_Home_Page/datacurrent.html on September, 14, 2026.

A few assumptions are made for the analysis and discussion:

- The weights of the first four strategies are annually rebalanced to keep the same distributions and comparability
- The target data fund is annually linearly rebalanced to gradually shift towards the desired terminal distribution
- Inflation, taxes and transaction costs are disregarded

4.1 Quantitative and Qualitative Analyse of the historical performances

The following code was generated by Claude Opus 5 to load the above referenced dataset, calculate the rolling 35-year cohorts for the five strategies, and visualise the distribution of annualized return and risk.

```
[161]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Setup
# Basic inputs for the retirement portfolio analysis

# Length of the savings horizon (a 30-year-old retiring at 65)
HORIZON = 35

# Rebalancing rule for the four fixed-weight portfolios
# True = reset to the target weight every year, False = buy and hold (weights
↳ drift with returns)
# Taxes and transaction costs are disregarded, which is defensible for a US
↳ retirement account
REBALANCE = True

# Start and end equity weight of the target date fund's glide path
GLIDE_FROM, GLIDE_TO = 0.90, 0.40
```

```

# Path to the Damodaran spreadsheet of annual returns since 1928
DATA_PATH = "Damodaran_hist_returns.xls"

# Load the data
# The table of annual returns starts in row 20 of the "Returns by year" sheet,
# so the 19 rows above it (title, links, premium calculator) are skipped
df = pd.read_excel(DATA_PATH, sheet_name="Returns by year", skiprows=19)

# Keep only the rows with a numeric year, which drops the summary rows at the
↳bottom
df = df[pd.to_numeric(df["Year"], errors="coerce").notna()].copy()
df["Year"] = df["Year"].astype(int)

# Rename the three series we need and keep them in decimals (they are not
↳percentages)
df = df.rename(columns={
    "Year": "year",
    "S&P 500 (includes dividends)": "stock",    # price appreciation plus
↳dividends
    "US T. Bond (10-year)": "bond",            # coupon plus price appreciation
    "3-month T.Bill": "bill",                  # risk-free rate, used for the
↳Sharpe ratio
})
df = df[["year", "stock", "bond", "bill"]].dropna().sort_values("year").
↳reset_index(drop=True)

```

```

[162]: # Define the strategies
# Each strategy is described by the equity weight it targets in every year of
↳the horizon

# The four fixed-weight portfolios hold the same target weight throughout
def fixed_path(w_equity, h=HORIZON):
    return np.full(h, w_equity)

# The target date fund moves linearly from 90% to 40% equity over the 35 years
def glide_path(h=HORIZON, w0=GLIDE_FROM, w1=GLIDE_TO):
    return np.linspace(w0, w1, h)

# The glide path only exists if the fund is traded back to its target every
↳year,
# so the target date fund is always rebalanced regardless of the REBALANCE
↳switch
STRATEGIES = {
    "100% Equity": dict(path=fixed_path(1.00), rebalance=REBALANCE),
    "80/20": dict(path=fixed_path(0.80), rebalance=REBALANCE),

```

```

    "60/40":          dict(path=fixed_path(0.60), rebalance=REBALANCE),
    "40/60":          dict(path=fixed_path(0.40), rebalance=REBALANCE),
    "Target date fund": dict(path=glide_path(),      rebalance=True),
}
ORDER = list(STRATEGIES.keys())

# Simulate one cohort
# Track the equity and bond sleeves separately so that the weights are allowed
↳to drift
def simulate(rs, rb, w_path, rebalance):
    h = len(rs)

    # Start with one dollar split according to the first target weight
    Ve, Vb = w_path[0], 1 - w_path[0]

    values = np.empty(h + 1)
    values[0] = 1.0
    weights = np.empty(h)

    for t in range(h):
        V = Ve + Vb

        # Trade back to the target weight at the start of the year, if
↳rebalancing
        if rebalance:
            Ve, Vb = w_path[t] * V, (1 - w_path[t]) * V

        # Record the weight the portfolio actually held during the year
        weights[t] = Ve / (Ve + Vb)

        # Let each sleeve grow at its own return
        Ve *= (1 + rs[t])
        Vb *= (1 + rb[t])
        values[t + 1] = Ve + Vb

    # Annual portfolio returns implied by the value path
    rets = values[1:] / values[:-1] - 1
    return values, rets, weights

# Largest peak-to-trough loss experienced along the way
def max_drawdown(values):
    peak = np.maximum.accumulate(values)
    return float((values / peak - 1).min())

# Run every cohort
# Using many cohorts avoids letting the market environment of one retirement
↳date drive the result

```

```

def run_cohorts(df, strategies=STRATEGIES, h=HORIZON):
    rows = []
    starts = df["year"].values[:len(df) - h + 1]

    for s0 in starts:
        i = df.index[df["year"] == s0][0]
        rs = df["stock"].values[i:i + h]
        rb = df["bond"].values[i:i + h]
        rf = df["bill"].values[i:i + h]

        for name, cfg in strategies.items():
            values, rets, weights = simulate(rs, rb, cfg["path"],
            ↪cfg["rebalance"])
            tw = values[-1]
            rows.append(dict(
                cohort_start = s0,
                cohort_end   = s0 + h - 1,
                strategy      = name,
                terminal      = tw,
                ↪dollar invested
                cagr          = tw ** (1 / h) - 1,
                ↪over the cohort
                vol           = rets.std(ddof=1),
                ↪annual returns
                sharpe        = (rets - rf).mean() / rets.std(ddof=1),
                max_dd        = max_drawdown(values),
                final_w_eq    = weights[-1],
                ↪by retirement
            ))
    return pd.DataFrame(rows)

# Summarise the distribution across cohorts.
# Terminal wealth is shown as a multiple of the dollar invested, the rest as
↪percentages
def summarise(res, transpose=True):
    g = res.groupby("strategy")
    out = pd.DataFrame({
        "Median wealth": g["terminal"].median().map("{:,.1f}x".format),
        "p10":          g["terminal"].quantile(0.10).map("{:,.1f}x".format),
        "p90":          g["terminal"].quantile(0.90).map("{:,.1f}x".format),
        "Worst":        g["terminal"].min().map("{:,.1f}x".format),
        "Best":         g["terminal"].max().map("{:,.1f}x".format),
        "Median return": g["cagr"].median().map("{:.1%}".format),
        "Worst return":  g["cagr"].min().map("{:.1%}".format),
        "Volatility":    g["vol"].median().map("{:.1%}".format),
        "Sharpe":        g["sharpe"].median().map("{:.3f}".format),
    })

```

```

        "Median drawdown": g["max_dd"].median().map("{:.1%}".format),
        "Worst drawdown": g["max_dd"].min().map("{:.1%}".format),
    }).loc[ORDER]
    out.index.name = None
    return out.T if transpose else out

# Share of cohorts in which each strategy ended up ahead of the all-equity
↳ portfolio
def win_rate(res, benchmark="100% Equity"):
    p = res.pivot(index="cohort_start", columns="strategy", values="terminal")
    share = p[ORDER].gt(p[benchmark], axis=0).mean()
    return share.map("{:.1%}".format).rename(f"beat {benchmark}").to_frame()

# Plot the results
def plot_all(res):
    from matplotlib.ticker import NullFormatter

    p = res.pivot(index="cohort_start", columns="strategy",
↳ values="terminal")[ORDER]

    fig, ax = plt.subplots(1, 2, figsize=(15, 5.5))

    # Spread of terminal wealth across all cohorts
    ax[0].boxplot([p[c] for c in ORDER], tick_labels=ORDER, showmeans=True)
    ax[0].set_ylabel(f"Terminal wealth per $1 after {HORIZON} years (log)")
    ax[0].set_title("Distribution across cohorts")
    ax[0].tick_params(axis="x", rotation=20)
    ax[0].grid(True, alpha=0.3)

    # The same outcomes ordered by the year the cohort started saving
    for c in ORDER:
        ax[1].plot(p.index, p[c], label=c, lw=1.4)
    ax[1].set_xlabel("Cohort start year")
    ax[1].set_ylabel("Terminal wealth per $1 (log)")
    ax[1].set_title("Outcome by retirement cohort")
    ax[1].legend(fontsize=8)
    ax[1].grid(True, alpha=0.3)

    # A log scale keeps equal proportional differences equally spaced
    def log_ticks(lo, hi):
        base = np.array([1, 1.5, 2, 3, 4, 5, 7])
        cand = np.concatenate([base * 10.0 ** e for e in range(-2, 4)])
        return cand[(cand >= lo * 0.95) & (cand <= hi * 1.05)]

    ticks = log_ticks(p.values.min(), p.values.max())
    for a in (ax[0], ax[1]):
        a.set_yscale("log")

```

```

        if len(ticks) >= 3:
            a.set_yticks(ticks)
            a.set_yticklabels([f"{t:g}x" for t in ticks])
            a.yaxis.set_minor_formatter(NullFormatter())

fig.tight_layout()

# Realised risk against realised return, one point per cohort
fig2, ax2 = plt.subplots(figsize=(9, 5))
for c in ORDER:
    sub = res[res.strategy == c]
    ax2.scatter(sub["vol"] * 100, sub["cagr"] * 100, s=12, alpha=0.6,
        label=c)
    ax2.set_xlabel("Realised volatility over the cohort (%)")
    ax2.set_ylabel("Realised annualised return (%)")
    ax2.set_title("Risk and return, one point per cohort")
    ax2.legend(fontsize=8)
    ax2.grid(True, alpha=0.3)

plt.show()

```

```

[163]: # Run the simulation over every cohort and summarise the outcomes
res = run_cohorts(df)

print(f"Data: {df.year.min()}--{df.year.max()} ({len(df)} years), "
      f"{res.cohort_start.nunique()} overlapping {HORIZON}-year cohorts, "
      f"rebalancing: {REBALANCE}")

summarise(res)

```

Data: 1928-2025 (98 years), 64 overlapping 35-year cohorts, rebalancing: True

```

[163]:

```

	100% Equity	80/20	60/40	40/60	Target date fund
Median wealth	39.0x	30.4x	21.3x	14.9x	25.9x
p10	27.9x	21.5x	13.7x	8.2x	15.7x
p90	49.8x	39.0x	31.8x	25.8x	32.8x
Worst	15.9x	13.0x	9.8x	6.8x	6.8x
Best	65.8x	44.5x	35.9x	28.4x	43.5x
Median return	11.0%	10.3%	9.1%	8.0%	9.7%
Worst return	8.2%	7.6%	6.7%	5.6%	5.6%
Volatility	17.2%	14.0%	11.0%	8.9%	12.2%
Sharpe	0.446	0.451	0.462	0.473	0.465
Median drawdown	-36.5%	-28.9%	-20.8%	-12.4%	-21.0%
Worst drawdown	-64.8%	-53.5%	-40.1%	-25.6%	-58.0%

```

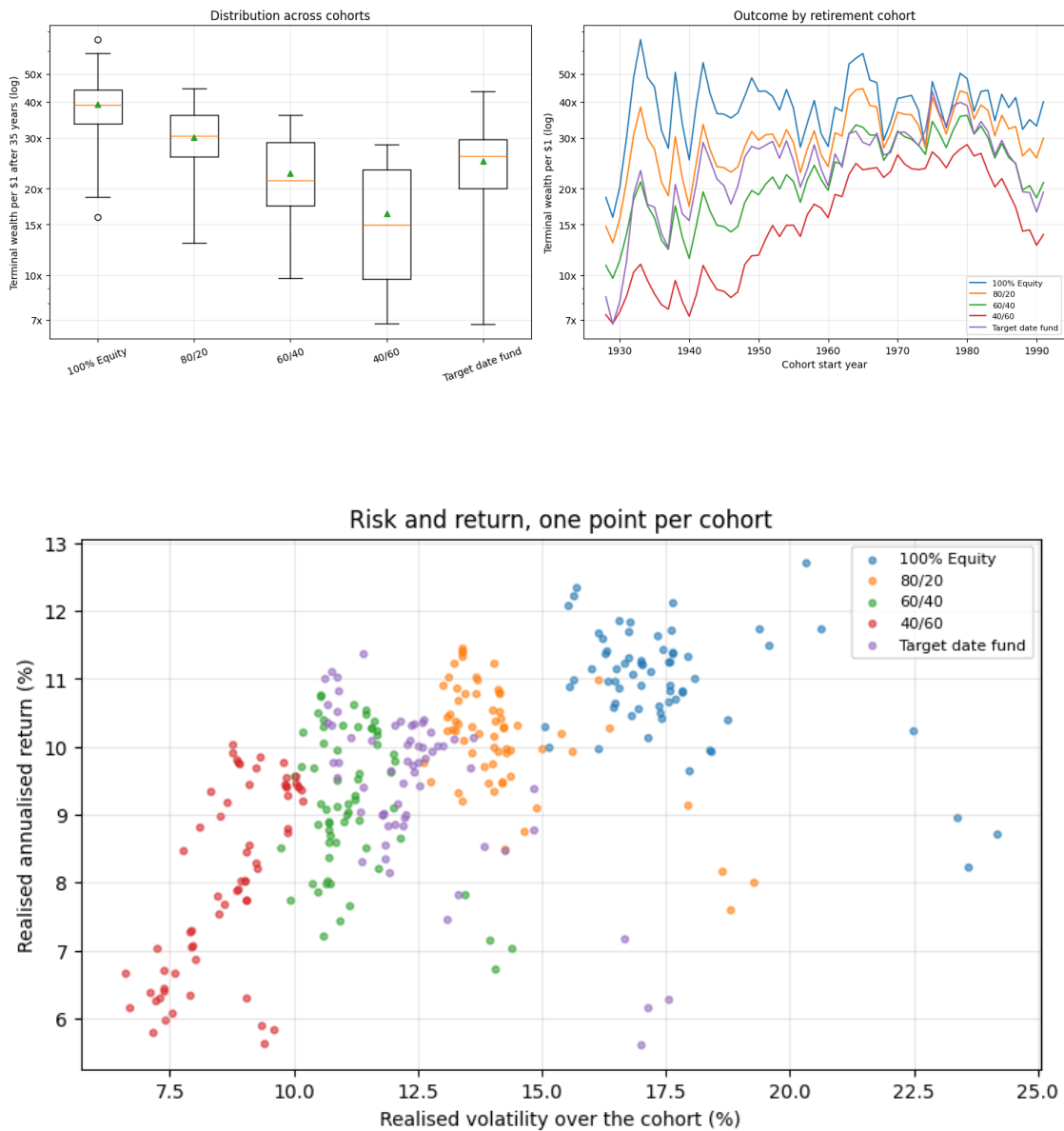
[164]: res.pivot(index="cohort_start", columns="strategy", values="terminal")[ORDER] \
        .idxmax(axis=1).value_counts().reindex(ORDER, fill_value=0) \
        .rename("cohorts won").to_frame()

```

```
[164]:
```

	cohorts won
100% Equity	62
80/20	0
60/40	0
40/60	0
Target date fund	2

```
[165]: plot_all(res)
```



Looking at the table, equities dominate on wealth multiples: a median of 39.0x, against 30.4x for the 80/20, 25.9x for the target date fund, 21.3x for the 60/40 and 14.9x for the 40/60 portfolio.

The second table shows that the all-equity portfolio achieved the highest terminal wealth in 62 of the 64 cohorts. Even the worst all-equity cohort, 1929 to 1963, still returned 15.9x, which exceeds the median outcome of the 40/60 portfolio.

At the same time equities showed the highest volatility of all strategies, at 17.2% against 8.9% for the 40/60, as well as the largest median drawdown of -36.5% and the largest single drawdown of -64.8%.

All in all this results in the Sharpe ratio improving monotonically as the bond weight is increased, from 0.446 for the all-equity portfolio to 0.473 for the 40/60 portfolio. The diversified portfolios are therefore more efficient on a risk-adjusted basis.
